

Radiant College, Manigram Butwal

**[ITEX 103] Object Oriented Programming
B Tech Ed. – 2ND SEMESTER**

Unit - I

(Introduction to OOP)

Arrays: Software Crisis and emergence of OOP, Procedural vs OOP Languages, Features of OOPs, Programming structures in C++, Basic programming terminologies: tokens, data types, operators, etc, Programming Construct, Control Structures

Procedural vs Object-Oriented Programming Language

- [1] What is Procedural Programming? Differentiate it from OOP. Also, write the usefulness of each paradigm.
- [2] Define Procedural Programming and Object-Oriented Programming. How are they different in terms of code organization?
- [3] Procedural Programming focuses on procedures while OOP focuses on objects." Justify this statement with examples.

Aspect	Procedural Programming (PP)	Object-Oriented Programming (OOP)
Primary Focus	Focuses on functions (procedures) that perform tasks.	Focuses on objects (data + methods) as basic units.
Data & Function Relation	Data and functions are separate	Data and functions are bundled together (encapsulation).
Data Security	Less secure (data is often global).	More secure (data is private/protected by default).
Code Reusability	Limited (reuses functions).	High (reuses classes via inheritance, composition).
Program Structure	Top-down approach (linear execution).	Bottom-up approach (objects interact with each other).
Polymorphism	Not supported.	Supported (method overloading/overriding).
Inheritance	Absent.	Supported (reuse/extend classes hierarchically).
Abstraction	No built-in abstraction mechanisms.	Uses abstract classes/interfaces for hiding complexity.
Example Languages	C, Pascal, FORTRAN.	C++, Java, Python, C#.
Maintenance	Harder to maintain for large projects.	Easier due to modularity and encapsulation.

Features of OOP

1. Encapsulation

- a. Bundles data (attributes) and methods (functions) into a single unit (class).
- b. Uses access modifiers (private, public, protected) for data hiding.

2. Inheritance

- a. Allows a class (derived) to inherit properties from another class (base).
- b. Promotes code reuse and hierarchical classification.
- c. Types: Single, Multiple, Multilevel, Hierarchical.

3. Polymorphism

- a. Enables "one interface, multiple implementations".
- b. Types:
 - i. Compile-time (method overloading)
 - ii. Runtime (method overriding)

4. Abstraction

- a. Hides complex details and shows only essential features.
- b. Implemented via abstract classes and interfaces.

5. Modularity

- a. Divides programs into independent, interchangeable modules (classes).
- b. Improves maintainability and collaboration.

6. Association

- a. Defines relationships between objects.
- b. Types:
 - i. Aggregation (weak "has-a" relationship)
 - ii. Composition (strong "part-of" relationship)

7. Dynamic Binding

- a. Resolves method calls at runtime rather than compile-time
- b. Enables runtime polymorphism.

8. Class and Object

- a. Class: Blueprint for creating objects.
- b. Object: Instance of a class with state and behavior.

9. Constructors/Destructors

- a. Constructor: A Special method to initialize objects.
- b. Destructor: Cleans up resources when the object is destroyed.

10. Exception Handling

- a. Treat errors as objects for more robust error management.
- b. Implemented via try-catch-throw mechanisms.

What was the Software Crisis?

The Software Crisis was a term coined in the late 1960s and 1970s to describe the difficulties in developing large-scale software systems. Key issues included:

- Cost overruns – Projects exceeded budgets.
- Time delays – Software took much longer to develop than expected.
- Poor quality – Bugs, inefficiencies, and unreliable systems.
- Unmaintainable code – Complex and unstructured programs were hard to modify.
- Scalability issues – Traditional procedural programming struggled with growing complexity.

Causes of the Software Crisis:

- Rapidly increasing hardware capabilities (software couldn't keep up).
- Lack of standardized methodologies for software development.
- Procedural programming limitations – Difficulty managing large codebases.
- Poor project management – Inadequate planning and estimation.

How Did OOP Help Solve the Crisis?

Object-Oriented Programming (OOP) emerged as a solution by introducing:

Feature	How It Helped
Encapsulation	Bundles data and methods into objects, reducing complexity.
Inheritance	Promotes code reuse, reducing redundancy.
Polymorphism	Allows flexible and extensible code.
Abstraction	Simplifies complex systems by hiding unnecessary details.
Modularity	Breaks programs into manageable, independent components.

Key OOP Languages That Emerged

- Simula (1967) – First OOP language.
- Smalltalk (1972) – Popularized OOP concepts.
- C++ (1985) – Brought OOP to systems programming.
- Java (1995) – Made OOP mainstream with portability.
- Python, C#, Ruby – Further expanded OOP adoption.

Impact of OOP

- Better code organization – Easier to maintain and scale.
- Increased productivity – Reusable components sped up development.
- Reduced bugs – Encapsulation prevented unintended data modifications.
- Foundation for modern software engineering – OOP principles influenced design patterns, frameworks, and best practices.

General Programming Structure of C++

```
#include <iostream> // Header file for input/output
using namespace std; // Using standard namespace

// Function declaration (modular structure)
int addNumbers(int a, int b);

// Class definition (OOP structure)
class Rectangle {
private:
    int length;
    int width;

public:
    // Constructor
    Rectangle(int l, int w) : length(l), width(w) {}

    // Method
    int calculateArea() {
        return length * width;
    }
};

int main() {
    // ===== SEQUENCE STRUCTURE =====
```

```

    cout << "Basic C++ Structures Demonstration\n\n";

    // ===== VARIABLES AND DATA TYPES =====
    int num1 = 5;        // Integer variable
    double num2 = 7.5;   // Floating-point variable
    char letter = 'A';   // Character variable
    bool flag = true;    // Boolean variable

    // ===== SELECTION STRUCTURE =====
    if (num1 > 0) {
        cout << num1 << " is positive\n";
    } else {
        cout << num1 << " is not positive\n";
    }

    // ===== ITERATION STRUCTURE =====
    cout << "Counting from 1 to 5:\n";
    for (int i = 1; i <= 5; i++) {
        cout << i << " ";
    }
    cout << "\n\n";

    // ===== FUNCTION CALL =====
    int sum = addNumbers(num1, 3);
    cout << "Sum of " << num1 << " and 3 is: " << sum << "\n";

    // ===== ARRAY USAGE =====
    int numbers[3] = {10, 20, 30};
    cout << "First element of array: " << numbers[0] << "\n";

    // ===== CLASS AND OBJECT =====
    Rectangle rect(4, 5);
    cout << "Area of rectangle (4x5): " << rect.calculateArea() <<
    "\n";

    return 0;
}

// Function definition

```

```
int addNumbers(int a, int b) {  
    return a + b;  
}
```

Basic Programming Terminologies

Tokens

Tokens are the smallest individual units in a program. In C++ (and similar languages), tokens are categorized as:

- **Keywords:** Reserved words with predefined meanings (e.g., `int`, `if`, `class`, `return`).
- **Identifiers:** User-defined names for variables, functions, classes, etc. (e.g., `sum`, `Rectangle`).
 - **Rules:** Must start with a letter or underscore, followed by letters, digits, or underscores; case-sensitive.
- **Literals:** Fixed values in the code.
 - **Integer:** Whole numbers (e.g., `5`, `-10`).
 - **Floating-point:** Decimal numbers (e.g., `3.14`, `-0.001`).
 - **Character:** Single characters (e.g., `'A'`, `'1'`).
 - **String:** Sequence of characters (e.g., `"Hello"`).
 - **Boolean:** `true` or `false`.
- **Operators:** Symbols that perform operations (e.g., `+`, `-`, `=`, `==`, `&&`).
- **Punctuators:** Symbols for syntax organization (e.g., `{`, `}`, `;`, `,`).

Data Types

Data types define the type and size of data a variable can hold. In C++, they are categorized as:

- **Primitive Data Types:**
 - **int:** Integer (e.g., `5`, size: 4 bytes).
 - **float, double:** Floating-point numbers (e.g., `3.14`, size: 4 and 8 bytes respectively).
 - **char:** Single character (e.g., `'A'`, size: 1 byte).

- **bool**: Boolean value (true or false, size: 1 byte).
- **Derived Data Types**
 - **Array**: Collection of elements of the same type (e.g., `int arr[5] = {1, 2, 3, 4, 5}`).
 - **Pointer**: Stores memory address (e.g., `int* ptr`).
 - **Reference**: Alias for another variable (e.g., `int& ref = num`).
- **User-Defined Data Types**
 - **Class/Structure**: Custom types for bundling data and methods (e.g., `class Rectangle { int length, width; }`).
 - **Enum**: Set of named integer constants (e.g., `enum Color {RED, GREEN, BLUE}`).
 - **Union**: Stores different data types in the same memory location

OPERATORS :

An operator is a symbol or letter which causes the compiler to take an action and yield a value. The operations being carried out are represented by operators. An operator acts on different data items/entities called Operands.

TYPES OF OPERATORS

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Ternary Operators
5. Increment and Decrement Operators

1. Arithmetic Operators

Arithmetic operators are those operators which perform arithmetic operations. Arithmetic operators are considered as basic operators and known as binary operators as they require two variables to be evaluated. For e.g. if we want to multiply two numbers, one has to enter or feed the multiplicand and the multiplier. That is why it is considered as a binary operator. There are mainly five types of arithmetic operators used in C language.

Operator	Meaning
----------	---------

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo(Remainder of an integer division)

Addition Operator(+)

```
int a = 4, b = 2, c;
```

```
c = a + b;
```

Output : c = 6

Subtraction Operator(-)

```
int a = 4, b = 2, c;
```

```
c = a - b;
```

Output : c = 2

Multiplication Operator(*)

```
int a = 4, b = 2, c;
```

```
c = a * b;
```

Output : c = 8

Division Operator(/)

```
int a = 5, b = 2, c;
```

```
c = a / b;
```

Output : c = 2

Modulus Operator(%)

```
int a = 5, b = 2, c;
```

```
c = a % b;
```

Output : c = 1

2. Relational Operators

Relational (Comparison) operators compare values to see if they are equal or if one of them is greater than or less than the other and so on. Comparison operators in C produce only a one or zero result. They are often specified as “true” or “false” respectively. The following operators are used to perform the relational or comparison operations of the two variables or expressions

Operator	Meaning
<	Less than
>	Greater than

< =	Less than or equal to
> =	Greater than or equal to
= =	Equal to
! =	Not equal to

The relational(comparison) operators are represented in the following syntax :

expression_1 relational_operator expression_2

For e.g.

Expression	Result
3 > 4	False
6 < = 2	False
10 > -32	True
(23 * 7) > = (-67 + 89)	True
8 = = 6	False
7 ! = 4	True

Difference between Equality (= =) and Assignment(=) Operators

The **Equality operator** = = is used to test check the equality of the given expression or a statement whereas **Assignment operator** = is used to assign the value of the variable or expression on the RHS to the LHS.

For example, the expression

value = = 3

tests whether value is equal to 3 ? . The expression has the boolean value true if the comparison is true and boolean false if it is false. But the expression

value = 3

assigns 3 to the value. The whole expression, in this case, has the value 3 because that's the value of the LHS.

3. Logical Operators

Logical operators are those operators which perform logical operations. There are three different types of logical operators used in C language :

- (a) **Logical AND**
- (b) **Logical OR**
- (c) **Logical NOT**

▪ **Logical AND operator (&&)**

The Logical AND operator is used in the following manner :

expression1 && expression2

The results of the logical AND operator are :

Situation	Results
True && True	True
True && False	False
False && True	False
False && False	False

For e.g.

if a = 4, b= 5 and c=6, then

(a < b) && (b<c) results in true because both expressions are true.

▪ Logical OR operator (||)

The Logical OR operator is used in the following manner :

expression1 || expression2

The results of the logical OR operator are :

Situation	Results
True True	True
True False	True
False True	True
False False	False

For e.g.

if a = 4, b= 5 and c = 6, then

(a < b) || (b>c) results in true because first expression is true and second is false.

▪ Logical NOT operator (!)

The Logical NOT operator is used in the following manner:

! (expression1)

The results of the logical operators are :

Situation	Results
! (True)	False
! (False)	True

For e.g.

! (4 > 6) results in true because (4>6) is false.

! (5 = 5) results in false because (5 = 5) is true.

4. Ternary Operators or Conditional Operators (?:)

C programming offers a Conditional operator (**?:**) that stores a value depending upon a condition. This operator is called **Ternary operator** i.e. it requires three expressions. The general form of the Ternary Operator **?:** is as follows :

Expression 1 ? Expression 2 : Expression 3

IF *Expression 1* evaluates to true i.e. 1, then the value of the whole expression is the value of the *Expression 2*, otherwise, the value of the whole expression is the value of the *Expression 3*. For Example:

max = (a > b) ? a : b ;

The value of '**a**' variable will be assigned to variable '**max**' if the test expression **a > b** evaluates to true otherwise the value of variable '**b**' will be assigned to variable '**max**'.

5. Increment (++) and Decrement (--)

C provides ++ and -- operators. These operators are called **increment** and **decrement** operators. The increment operator ++ adds 1 to its operand.

++ x means x = x + 1

x ++ means x = x + 1

-- x means x = x - 1

x -- means x = x - 1

These operators exist in 2 types

(i) **Prefix**

(ii) **Postfix**

Prefix means before its operand. For example

++ x

-- x

Postfix means after its operand. For example

x ++

x --

Prefix increment or decrement operators (For e.g. ++x or --x) follow the FIRST CHANGE THEN USE rule. They first change the value of their operand and then use a new value for the expression. For Example take the following expression :

int x,y = 5;

x = ++y;

In this example, in the first statement the variable y is initialized with value 5. In the second statement, this value of y is first incremented by 1 and then the incremented value of y is assigned to x. After the second statement, the values of x and y are

```
x = 6;  
y = 6;
```

Postfix increment or decrement operators (For e.g. x++ or x--) follow the FIRST USE THEN CHANGE rule. They first use the value of the operand in the expression and then increment or decrement the value of the operand. For Example take the following expression :

```
int x,y = 5;  
x = y++;
```

In this example, in the first statement the value of y is assigned as 5. In the second statement, this value of y is first assigned to x and then the value of y is incremented by 1. After the second statement, the values of x and y are

```
x = 5;  
y = 6;
```

• TYPE CONVERSION:

The process of converting one predefined type into another is called **Type conversion**. When constants and variables of different types are mixed in an expression, they are converted to the same type. There are two types of type conversions:

- (a) Implicit type conversion
- (b) Explicit type conversion

An **Implicit type conversion** is a conversion performed by the compiler without programmer's intervention. An implicit conversion is applied generally whenever data types are intermixed in an expression, so as not to lose information. The compiler converts all operands upto the type of the largest operand, which is called **type promotion**. This is done operation by operation in this order : double, float, long, int, char . For e.g.

```
float a=5.5,c;  
int b=3;  
c=a+b;          // implicit type conversion  
printf("c : %.2f",c);
```

An **Explicit type conversion** is user-defined that forces an expression to be of specific type. The explicit conversion of an operand to a specific type is called **Type casting**. Type casting in C is done as shown below :

(datatype) expression

where type is a valid C data type to which the conversion is to be done. For example, to make sure that the expression $(x + y/2)$ evaluates to type float, write it as:
 $(\text{float}) (x + y/2)$

```
float pi;  
pi=(float)22/7;    // explicit type conversion  
printf("pi : %.2f",pi);
```

• DECISION MAKING STATEMENTS :

A C program is a set of statements which are normally executed sequentially in the order in which they appear. This happens when no options or no repetitions of certain calculations are necessary. However, in practice, we have a number of situations where we may have to change the order of execution of statements based on certain conditions, or repeat a group of statements until certain specified conditions are met. This involves a kind of decision making to see whether a particular condition has occurred or not and then direct the computer to execute certain statements accordingly. C language must be able to perform different sets of actions depending upon the circumstances. C has two major decision making (conditional) statements:

- (i) ***The If-else statement***
- (ii) ***The Switch statement***

IF ELSE STATEMENT

There are mainly 3 types of If Else Statements used in C :

1. **Simple If Statement**
2. **If Else Statement**
3. **Nested If Else Ladder (if else if statement)**

(1) Simple If Statement

The general form of simple If statement is:

```
if (condition)
{
    Statement-block;
}
Statement-x;
```

The statement block may be a single statement or a group of statements. If the condition is true, the statement-block will be executed and then the execution will jump to the statement-x. If the condition is false, the statement-block will be skipped and the execution will directly jump to the statement-x. For example,

```
int a=5;
if(a==5)
```



```
{  
    printf("%d",a);  
}
```

In above simple if statement, the printf statement inside if block is executed because the value of a is initialized as 5 and the condition a==5 inside if is true. Take another example,

```
int a=8;  
if(a==5)  
{  
    printf("%d",a);  
}
```

In above simple if statement, the printf statement inside if block is skipped because the value of a is initialized as 8 and the condition a==5 inside if is false.

Program 1 : WAP to display the greatest out of three numbers.

```
#include<stdio.h>  
#include<conio.h>  
void main()  
{  
    int a,b,c,max;  
    clrscr();  
    printf("Enter three numbers : ");  
    scanf("%d%d%d",&a,&b,&c);  
    max=a;  
    if(b>max)  
        max=b;  
    if(c>max)  
        max=c;  
    printf("\nLargest number is : %d",max);  
    getch();  
}
```

(2) If Else Statement

The if else statement is an extension of the simple if statement. The general form of if else statement is :

```
if (condition)  
{  
    true-block statement(s);
```

```
    }  
    else  
    {  
        false-block statement(s);  
    }  
    statement-x;
```

In the above syntax, if the condition is true, then the true-block statement(s), immediately following the if statements are executed, otherwise the false-block statement(s) are executed. In either case, either true-block or false-block will be executed. In both the cases, the control is transferred subsequently to statement-x. For example,

```
if(a>b)  
{  
    printf("%d is greater",a);  
}  
else  
{  
    printf("%d is greater",b);  
}
```

In above example, if the condition $a > b$ inside if is true, then the printf statement inside if block is executed and the else block statement is skipped. If the condition $a > b$ is false, then the printf statement inside if block is skipped and the else block printf statement is executed.

Program 2 : WAP to display the greatest out of two numbers.

```
#include<stdio.h>  
#include<conio.h>  
void main()  
{  
    clrscr();  
    int a,b;  
    printf("Enter two numbers : ");  
    scanf("%d%d",&a,&b);  
  
    if(a>b)  
        printf("\n%d is greater than %d",a,b);  
    else
```

```
    printf("\n%d is greater than %d",b,a);  
    getch();  
}
```

Program 3 : WAP to accept the age of a person and display whether the person is eligible to vote or not ?

```
#include<stdio.h>  
#include<conio.h>  
void main()  
{  
    clrscr();  
    int age;  
    printf("Enter age : ");  
    scanf("%d",&age);  
    if(age>=18)  
        printf("\nPerson is eligible to vote ");  
    else  
        printf("\nPerson is not eligible to vote ");  
    getch();  
}
```

Program 4 : WAP to accept an integer from keyboard and display whether the given number is even or odd ?

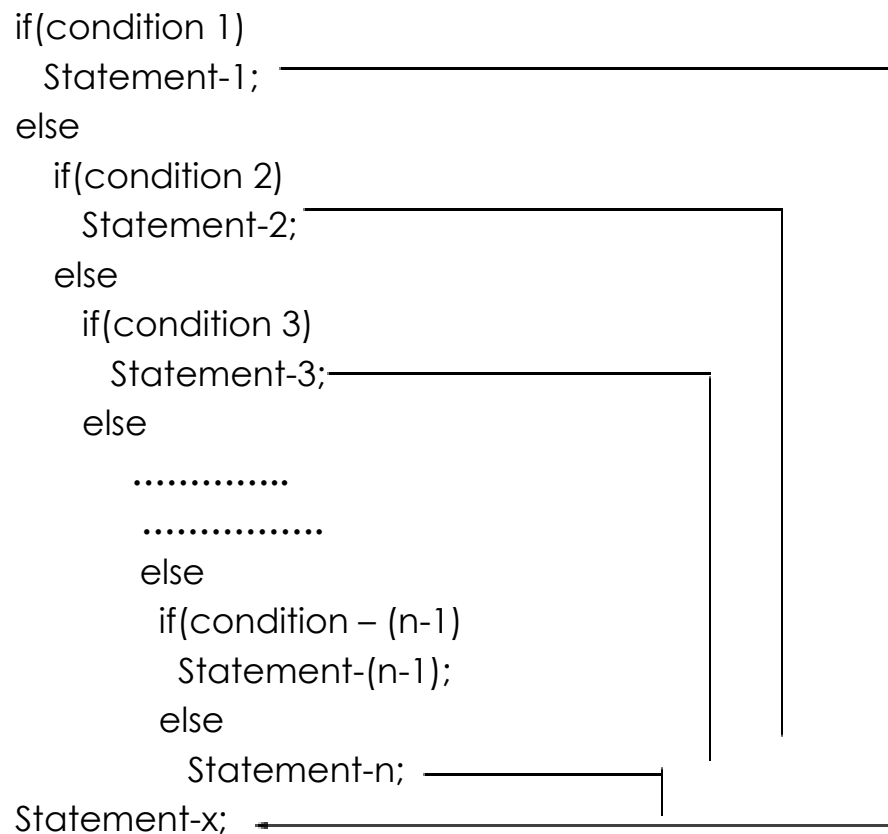
```
#include<stdio.h>  
#include<conio.h>  
void main()  
{  
    clrscr();  
    int n;  
    printf("Enter the value of n : ");  
    scanf("%d",&n);  
    if(n%2==0)  
        printf("\n%d is even",n);  
    else  
        printf("\n%d is odd",n);  
    getch();  
}
```

Program 5 : WAP to accept an year from the keyboard and display whether the given year is a leap year or not ?

```
#include<stdio.h>
#include<conio.h>
void main()
{
clrscr();
int yr;
printf("Enter the year : ");
scanf("%d",&yr);
if(yr%4==0)
printf("\n%d is leap year",yr);
else
printf("\n%d is not leap year ",yr);
getch();
}
```

(3) Nested Else-If Ladder :

There is another way of putting ifs together when multipath decisions are involved. A multipath decision is a chain of ifs in which the statement associated with each else is an if. The general form of a **Nested if else Ladder** is :



The construct is known as the else if ladder. The conditions are evaluated from the top (of the ladder), downwards. As soon as a true condition is found, the statement associated with it is executed and the control is transferred to the statement-x (skipping the rest of the ladder). When all the (n-1) conditions become false, then the final else containing the Statement-n will be executed. There is no limit of nested else statements in nested If-Else Ladder. Let us consider an example of grading the students in an academic institution. The grading is done according to the following rules:

Average marks	Grade
80 to 100	Honours
60 to 79	First division
50 to 59	Second division
40 to 49	Third division
0 to 39	Fail.

The grading is done using the else if ladder as follows:

```
if (marks >=80 )
    printf("Honours");
else
    if (marks >= 60)
        printf("First division");
    else
        if (marks >= 50)
            printf("Second division");
        else
            if (marks >= 40)
                printf("Third division");
            else
                printf("Fail");
```

Program 6 : WAP to accept a day number [1-7] and display its equivalent day. For example, 1 for Sunday, 2 for Monday,....., 7 for Saturday.

```
#include<stdio.h>
#include<conio.h>
void main()
{
    clrscr();
    int dayno;
    printf("Enter day number [1/2/3/4/5/6/7] : ");
    scanf("%d",&dayno);
    printf("\n");
```

```
if(dayno==1)
    printf("Sunday");
else
    if(dayno==2)
        printf("Monday");
    else
        if(dayno==3)
            printf("Tuesday");
        else
            if(dayno==4)
                printf("Wednesday");
            else
                if(dayno==5)
                    printf("Thursday");
                else
                    if(dayno==6)
                        printf("Friday");
                    else
                        if(dayno==7)
                            printf("Saturday");
                        else
                            printf("Invalid day number entered");
getch();
}
```

Program 7 : WAP to display the result of a student acc. to grades given below :

Grade	Result
A	First class with distinction
B	First class
C	Second class
D	Conditionally passed
For all other grades	Failed

```
#include<stdio.h>
#include<conio.h>
```

```
void main()
{
    clrscr();
    char grade;
    printf("Enter the grade [A/B/C/D] :");
    scanf("%c",&grade);
```

```
printf("\n");
if(grade=='A')
    printf("First class with distinction");
else
    if(grade=='B')
        printf("First class");
    else
        if(grade=='C')
            printf("Second class");
        else
            if(grade=='D')
                printf("Conditionally passed");
            else
                printf("Failed");
getch();
}
```

Program 8 : WAP to display the roots of a quadratic equation and also display whether the roots are real and equal, real and unequal or complex and imaginary.

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
void main()
{
    clrscr();
    float a,b,c,d,r1,r2;
    printf("Enter the coefficient's a,b,c of a quadratic equation : ");
    scanf("%f%f%f",&a,&b,&c);
    d=((b*b)-(4*a*c));
    if(d==0)
    {
        r1=r2=-b/(2*a);
        printf("\nRoot-1 : %.2f\nRoot-2 : %.2f",r1,r2);
        printf("\nRoots are real and equal");
    }
    else
        if(d>0)
        {
            r1=(-b+sqrt(d))/(2*a);
            r2=(-b-sqrt(d))/(2*a);
            printf("\nRoot-1 : %.2f\nRoot-2 : %.2f",r1,r2);
            printf("\nRoots are real and unequal");
        }
}
```

```
}  
else  
    printf("\nRoots are complex and imaginary");  
getch();  
}
```

SWITCH STATEMENT

C has a built-in multiway decision statement known as a **switch**. The switch statement tests the value of a given variable (or expression) against a list of case values and when a match is found, a block of statements associated with that case is executed. The general form of the switch statement is as shown below:

```
switch(variable or expression)  
{  
    case constant 1 : statement 1;  
                    break;  
    case constant 2 : statement 2;  
                    break;  
  
    case constant 3 : statement 3;  
                    break;  
  
    .....  
    .....  
    case constant n-1 : statement n-1;  
                    break;  
    default          : statement n;  
}  
statement-x;
```

The variable or expression is integer or character variable or expression. Constant 1, Constant 2 are integer or character constants and are known as case labels. Each of these constant should be unique within a switch

statement. Statement 1, statement 2.....are statement lists and may contain zero or more statements. There is no need to put braces around these blocks. Note that case labels end with a colon (:).

When the switch is executed, the value of the variable or expression is successfully compared against the values constant 1, constant 2,..... . If a match is found whose value matches with the value of the variable or expression, then the block of statement(s) that follows the case are executed.

The **break** statement at the end of each block signals the end of a particular case and causes an exit from the switch statement, transferring the control the statement-x following the switch.

The **default** is an optional case. When present, it will be executed if the value of the variable or expression does not match with any of the case constants. If not present, no action takes place if all matches fail and the control goes to statement-x.

Program 1 : WAP to accept a day number [1-7] and display its equivalent day

```
#include<stdio.h>
#include<conio.h>
void main()
{
clrscr();
int dayno;

printf("Enter day number [1/2/3/4/5/6/7] : ");
scanf("%d",&dayno);
printf("\n");
switch(dayno)
{
case 1 : printf("Sunday");
        break;
case 2 : printf("Monday");
        break;
case 3 : printf("Tuesday");
        break;
case 4 : printf("Wednesday");
        break;
case 5 : printf("Thursday");
        break;
case 6 : printf("Friday");
        break;
case 7 : printf("Saturday");
```

```
        break;
default : printf("Invalid day no. entered");
}
getch();
}
```

Program 2 : WAP to accept the grade of a student and display the result of student acc. to conditions given below :

GRADE	RESULT
A	First class with distinction
B	First class
C	Second class
D	Conditionally passed
For all other grades	Failed

```
#include<stdio.h>
#include<conio.h>
void main()
{
clrscr();
char grade;
printf("Enter grade [A/B/C/D] : ");
scanf("%c",&grade);
printf("\n");
switch(grade)
{
case 'A' : printf("First class with distinction");
           break;
case 'B' : printf("First class");
           break;
case 'C' : printf("Second class");
           break;
case 'D' : printf("conditionally passed");
           break;
default : printf("Failed");
}
getch();
}
```

Program 3 : WAP to accept an alphabet from the keyboard and display whether it is a vowel or a consonant

```
#include<stdio.h>
```

```
#include<conio.h>
void main()
{
clrscr();
char alpha;
printf("Enter an alphabet : ");
scanf("%c",&alpha);
printf("\n");
switch(alpha)
{
case 'A' :
case 'E' :
case 'I' :
case 'O' :
case 'U' :
case 'a' :
case 'e' :
case 'i' :
case 'o' :
case 'u' : printf("%c is a Vowel",alpha);
           break;
default : printf("%c is a Consonant",alpha);
}
getch();
}
```

Program 4 : WAP to accept two numbers and also accept one operator [+ , - , * , / , %] . Display the arithmetic operation of numbers acc. to arithmetic operator entered by the user.
(M. Imp.)

```
#include<stdio.h>
#include<conio.h>
void main()
{
clrscr();
int a,b;
char op;
printf("Enter two numbers :");
scanf("%d%d",&a,&b);
printf("Enter operator[+,-,*,/,%] :");
fflush(stdin);
scanf("%c",&op);
printf("\n");
```

```
switch(op)
{
case '+' : printf("Sum of %d and %d is : %d",a,b,a+b);
           break;
case '-' : printf("Difference of %d and %d is : %d ",a,b,a-b);
           break;
case '*' : printf("Product of %d and %d is : %d",a,b,a*b);
           break;
case '/' : if(b==0)
            printf("Divide by zero error. Enter divisor other than zero");
            else
            printf("Quotient of %d and %d is : %d",a,b,a/b);
            break;
case '%': if(b==0)
            printf("Divide by zero error. Enter divisor other than zero");
            else
            printf("Remainder of %d and %d is : %d",a,b,a%b);
            break;
default  : printf("Invalid operator");
}
getch();
}
```

Program 5 : WAP to display the area acc. to choices given below :

- 1. Area of a rectangle**
- 2. Area of a circle**
- 3. Area of a square**

```
#include<stdio.h>
#include<conio.h>
void main()
{
clrscr();
int choice;
printf("  AREA'S");
printf("\n-----");
printf("\n1. Rectangle");
printf("\n2. Circle ");
printf("\n3. Square ");
printf("\n-----");
printf("\nEnter your choice [1/2/3] : ");
```

```

scanf("%d",&choice);
printf("\n");
switch(choice)
{
case 1 : int l,b,ar1;
        printf("Enter length and breath of rectangle : ");
        scanf("%d%d",&l,&b);
        ar1=l*b;
        printf("\nArea of rectangle is :%d",ar1);
        break;
case 2 : float rad,ar2;
        printf("Enter radius of circle :");
        scanf("%f",&rad);
        ar2=((float)22/7)*rad*rad;
        printf("\nArea of circle is :%.2f",ar2);
        break;
case 3 : int len,ar3;
        printf("Enter length of square :");
        scanf("%d",&len);
        ar3=len*len;
        printf("\nArea of square is :%d",ar3);
        break;
default : printf("\nInvalid option");
}
getch();
}

```

Difference between switch-case and multiway if-else structure

Switch	If-else
switch can only test for equality.	if-else can evaluate a relational or logical expression i.e. multiple conditions.
The switch statement selects its branches by testing the value of same variable(against a set of constants).	if-else construction lets you use a series of expressions that may involve unrelated variables and complex expressions.

- 1) **Discuss with examples the various control statements available in C.**
Explain the various types of loops and their applications.
What is the difference between break and continue statements? Explain using suitable examples.

Ans. CONTROL STATEMENTS IN C :

Various control statements available in C are:

- (1) simple if statement
- (2) if else statement
- (3) nested if else ladder (if else if)
- (4) switch statement
- (5) while loop
- (6) for loop
- (7) do while loop
- (8) break statement

(9) continue statement

(10) goto statement

(1) Simple If Statement

The general form of simple If statement is:

```
if (condition)
{
    Statement-block;
}
Statement-x;
```

The statement block may be a single statement or a group of statements. If the condition is true, the statement-block will be executed and then the execution will jump to the statement-x. If the condition is false, the statement-block will be skipped and the execution will directly jump to the statement-x. For example,

```
int a=5;
if(a==5)
{
    printf("%d",a);
}
```

In above simple if statement, the printf statement inside if block is executed because the value of a is initialized as 5 and the condition a==5 inside if is true. Take another example,

(2) If Else Statement

If else statement is an extension of the simple if statement. The general form of if else statement is:

```
if (condition)
{
    true-block statement(s);
}
else
{
    false-block statement(s);
}
```

```

    false-block statement(s);
}
statement-x;

```

In the above syntax, if the condition is true, then the true-block statement(s), immediately following the if statements are executed, otherwise the false-block statement(s) are executed. In either case, either true-block or false-block will be executed. In both the cases, the control is transferred subsequently to statement-x. For example,

```
if(a>b)
{
    printf("%d is greater",a);
}
else
{
    printf("%d is greater",b);
}
```

In above example, if the condition `a>b` inside `if` is true, then the `printf` statement inside `if` block is executed and the `else` block statement is skipped. If the condition `a>b` is false, then the `printf` statement inside `if` block is skipped and the `else` block `printf` statement is executed.

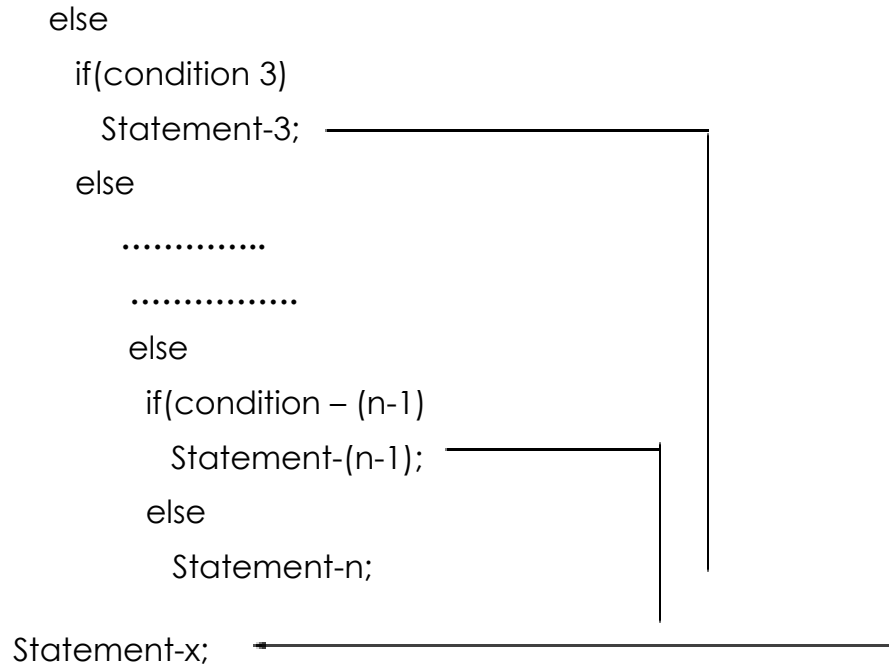
(3) Nested Else-If Ladder :

There is another way of putting ifs together when multipath decisions are involved. A multipath decision is a chain of ifs in which the statement associated with each else is an if. The general form of a **Nested if else Ladder** is :

```

if(condition 1)
    Statement-1;
else
    if(condition 2)
        Statement-2;

```

The construct is known as the else if ladder. The conditions are evaluated from the top (of the ladder), downwards. As soon as a true condition is found, the statement associated with it is executed and the control is transferred to the statement-x (skipping the rest of the ladder). When all the (n-1) conditions become false, then the final else containing the Statement-n will be executed. There is no limit of nested else statements in nested If-Else Ladder. Let us consider an example of grading the students in an academic institution. The grading is done according to the following rules:

Average marks	Grade
80 to 100	Honours
60 to 79	First division
50 to 59	Second division
40 to 49	Third division
0 to 39	Fail.

The grading is done using the else if ladder as follows:

```

if (marks >=80 )
  printf("Honours");

```

```
else
    if (marks >= 60)
        printf("First division");
    else
        if (marks >= 50)
            printf("Second division");
        else
            if (marks >= 40)
                printf("Third division");
            else
                printf("Fail");
```

(4) Switch statement :

C has a built-in multiway decision statement known as a switch. The switch statement tests the value of a given variable (or expression) against a list of case values and when a match is found, a block of statements associated with that case is executed. The general form of the switch statement is as shown below:

```
switch(variable or expression)
{
    case constant 1 : statement 1;
                    break;
    case constant 2 : statement 2;
                    break;

    case constant 3 : statement 3;
                    break;
    .....
    .....
    case constant n-1 : statement n-1;
                    break;
    default          : statement n;
}
statement-x;
```

The variable or expression is integer or character variable or expression. Constant 1, Constant 2 are integer or character constants and are known as case labels. Each of these constant should be unique within a switch statement. Statement 1, statement 2.....are statement lists and may contain zero or more statements. There is no need to put braces around these blocks. Note that case labels end with a colon (:).

When the switch is executed, the value of the variable or expression is successfully compared against the values constant 1, constant 2,..... . If a match is

found whose value matches with the value of the variable or expression, then the block of statement(s) that follows the case are executed.

The **break** statement at the end of each block signals the end of a particular case and causes an exit from the switch statement, transferring the control the statement-x following the switch.

The **default** is an optional case. When present, it will be executed if the value of the variable or expression does not match with any of the case constants. If not present, no action takes place if all matches fail and the control goes to statement-x.

LOOPING:

There are some situations in C programming in which a particular set of statements are repeated again and again. To avoid the repetition of the statements, C allows a set of instructions to be performed repeatedly until a certain condition is fulfilled by using **looping**. The main advantage of using loops in C is that it reduces program size and complexity of the program. Looping consists of mainly three parts:

- (c) *Initialization*
- (d) *Test Condition*
- (e) *Incrementer or Decrementor*

Accordingly, there are three types of loops available in C.

- (i) **while loop**
- (ii) **for loop**
- (iii) **do-while loop**

(5) **while statement :**

The simple of all the looping statements in C is the while statement. While loop is entry-controlled loop i.e. the condition is test before executing the body of the loop. The basic syntax of the while statement is:

```
initialization;
while(test condition)
{
    statement(s);
    incrementor/decrementor;
}
```

- a) *Initialization* of the control variables is done first, using assignment statements such as $i = 1$ or $\text{count} = 0$. The variables i and count are loop control variables.
- b) The value of the control variable is tested using the *test-condition*. The test-condition is a relational expression, such as $i \leq 10$ that determines when the

loop will exit. If the condition is true, the body of the loop is executed, otherwise the loop is terminated and the execution continues with the statement that immediately follows the loop.

- c) When the body of the loop is executed, the statement(s) are executed first and then the control variable is *incremented* or *decremented* using an assignment statement such as `i++` (or `i=i+1`) or `j--` (or `j=j+1`) and the new value of control variable is again tested to see whether it satisfies the loop condition. If the condition is satisfied, the body of the loop is again executed. This process continues till the value of the control variables fails to satisfy the test-condition.

For example:

```
i=1;
while(i<=10)
{
    printf("%d\t", i);
    i++;
}
```

Output :

1 2 3 4 5 6 7 8 9 10

(6) for statement :

The for loop is another entry-controlled loop that provides a more concise loop control structure. The general form of for loop is:

```
for (initialization; test-condition; incrementor/decrementor)
{
    statement(s);                                // body of the loop
}
```

The working of while and for loop is same. (Please note from (a), (b) and (c) of while loop)

For example :

```
for(i=1 ; i<=10 ; i++)
{
    printf("%d\t", i);
}
```

Output :

1 2 3 4 5 6 7 8 9 10

(7) do while loop :

The while loop construct makes a test of condition before the loop is executed. Therefore, the body of the loop may not be executed at all if the conditions is not satisfied at the very first attempt. On some occasions it might be necessary to execute the body of the loop before the test is performed. Such situations can be handled with the help of the do while loop. The syntax of do while loop is :

```

initialization;
do
{
    statement(s);
    incrementor/decrementor;
}while(test-condition);

```

On reaching the do statement, the program proceeds to evaluate the body of the loop first. At the end of the loop, the test-condition in the while statement is evaluated. If the condition is true, the program continues to evaluate the body of the loop once again. This process continues as long as the condition is true. When the condition becomes false, the loop will be terminated and the control goes to the statement that appears immediately after the while statement.

Since the test-condition is evaluated at the bottom of the loop, the do..while construct provides an exit-controlled loop and therefore the body of the loop is always executed at least once.

For example:

```

do
{
    printf("\nInput a number : ");
    scanf("%d",&n);
}while(n>0);

```

The above statements of the program reads a number from the keyboard until a zero or a negative number is accepted from the keyboard.

(8) Break statement :

The **break** statement enables a program to skip over part of the code. A break statement terminates the smallest enclosing while, do-while, for or switch statement. A break within a loop should always be protected within an if statement which provides the test to control the exit condition. For example:

```

for (i=1 ; i<=10 ; i++)
{
    if(i ==5)
    {
        break;
    }
}

```

```

    }
    printf("%d\t",i);
}
printf("\nOut of loop");

```

A break statement skips the rest of the loop and jumps over to the statement following the loop. So the output of the above program is :

```

1      2      3      4
Out of loop

```

(9) Continue statement :

The **continue** statement is another jump statement like the break statement as both the statements skip over a part of the code. But the difference between the break statement and continue statement is that the continue statement skips the rest of the loop statements and causes the next loop statements to be executed. For example:

```

for (i=1 ; i<=10 ; i++)
{
    if(i ==5)
    {
        continue;
    }
    printf("%d\t",i);
}
printf("\nOut of loop");

```

A continue statement skips the rest of the loop statements for 5th iteration when i=5 and jumps directly to the next iteration i.e. i = 6.. So the output of the above program is :

```

1      2      3      4      6      7      8      9      10
Out of loop

```

(10) Go To statement :

The **goto** statement is used to change the program execution sequence by transferring the control to some other part of the program. The general syntax of the goto statement is :

```
goto label;
```

where label is a valid C++ identifier used to label the destination such that control could be transferred. For example :

```

void main()
{
    clrscr();
    int a,b;
    printf("Enter two numbers : ");
    scanf("%d%d",&a,&b);
    if(a>b)

```

```

        goto output1;
    else
        goto output2;
    output1 :
        printf("Largest value : %d",a);
        goto stop;

    output2 :
        printf("Largest value : %d",b);
        goto stop;
    stop :
        getch();
}

```

2) Differentiate between while and do-while loop using suitable examples.[10]
[May-2013]

Ans.

	While loop	For loop	Do while loop
1.	Entry-controlled loop	Entry-controlled loop	Exit-controlled loop.
2.	It makes the test of the condition before the loop is executed.	It makes the test of the condition before the loop is executed.	It makes the test of the condition after the loop is executed. It means that atleast for the 1 st iteration of the loop, the body of the loop is executed without testing the condition. So the do while loop executes atleast once.
3.	0 or more iterations	0 or more iterations.	1 or more iterations.
4.	Not suitable for menu driven programs.	Not suitable for menu driven programs.	Suitable for menu driven programs.
5.	Syntax: initialization; while(condition) { statement(s); incrementor/decrementor; }	Syntax: for(initialization;condition;inc./dec.) { statement(s); }	Syntax: initialization; do { statement(s); incrementor/decrementor; } while(condition);

● INCREMENT (+ +) AND DECREMENT (- -) OPERATORS

C provides ++ and -- operators. These operators are called **increment** and **decrement** operators. The increment operator ++ adds 1 to its operand.

++ x means $x = x + 1$

x ++ means $x = x + 1$

-- x means $x = x - 1$

x -- means $x = x - 1$

These operators exist in 2 types

(i) **Prefix**

(ii) **Postfix**

Prefix means before its operand. For example

++ x

-- x

Postfix means after its operand. For example

x ++

x --

Prefix increment or decrement operators (For e.g. ++x or --x) follow the FIRST CHANGE THEN USE rule. They first change the value of their operand and then use a new value for the expression. For Example take the following expression :

```
int x,y = 5;
```

```
x = ++y;
```

In this example, in the first statement the value of y is assigned as 5. In the second statement, this value of y is first incremented by 1 and then the incremented value of y is assigned to x. After the second statement, the values of x and y are

```
x = 6;
```

```
y = 6;
```

Postfix increment or decrement operators (For e.g. x++ or x--) follow the FIRST USE THEN CHANGE rule. They first use the value of the operand in the expression and then increment or decrement the value of the operand. For Example take the following expression :

```
int x,y = 5;
```

```
x = y++;
```

In this example, in the first statement the value of y is assigned as 5. In the second statement, this value of y is first assigned to x and then the value of y is incremented by 1. After the second statement, the values of x and y are

```
x = 5;
```

```
y = 6;
```

Program : Increment and Decrement operators

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main( )
```

```
{
```

```
clrscr( );
```

```
int a=10,b=20;
```

```
printf("Original values \n a : %d \t b : %d",a,b);
```

```
printf("\n\n Prefix operators (First Change then use) :\n");
```



```
printf("\n++a : %d",++a);
printf("\n--b : %d",--b);
a=10;
b=20;
printf("\n\n Postfix operators (First use then Change) :\n");
printf("\na++ : %d",a++);
printf("\nb-- : %d",b--);
printf("\n\na : %d\tb : %d",a,b);
getch( );
}
```

- 3) **How many times while loop executes? Justify. Also specify the value of a after every iteration of the loop:**

```
int x=65, y=45, a=1;
while(a<9)
{
    if(x>y)
    {
        x=x-5;
        a=a-2;
    }
    a=a+1;
}
```

Ans. (a) This while loop executes 16 times.

- (b) Iteration – 1 : a = 0
Iteration – 2 : a = -1
Iteration – 3 : a = -2
Iteration – 4 : a = -3
Iteration – 5 : a = -2
Iteration – 6 : a = -1
Iteration – 7 : a = 0
Iteration – 8 : a = 1
Iteration – 9 : a = 2
Iteration – 10 : a = 3
Iteration – 11 : a = 4
Iteration – 12 : a = 5
Iteration – 13 : a = 6
Iteration – 14 : a = 7
Iteration – 15 : a = 8

Iteration – 16 : a = 9

4) Find the output of the following correct program. Justify the output & print it.

```
#include<stdio.h>
main()
{
    int n, r;

    for(n=5;n>=1;n - -)
    {
        switch(n)
        {
            case 5 : r = n%5;
                    break;
            case 3 : r = (n+r)%10;
                    break;
            case 1 : r = (n * n)/2;
                    break;
            default : r = n * 8;
                    break;
        }
        printf("%d", r);
    }
}
```

Ans. 0 32 5 16 0

- (a) Iteration – 1 : when n = 5, case 5 will be executed and the statement
 $r = n \% 5$; evaluates to $r = 5 \% 5 = 0$
 i.e. $r = 0$
- (b) Iteration – 2 : when n = 4, default case will be executed and the statement
 $r = n * 8$; evaluates to $r = 4 * 8 = 32$
 i.e. $r = 32$
- (c) Iteration – 3 : when n = 3, case 3 will be executed and the statement
 $r = (n+r) \% 10$; evaluates to $r = (3+32) \% 10 = 5$
 i.e. $r = 5$
- (d) Iteration – 4 : when n = 2, default case will be executed and the statement
 $r = n * 8$; evaluates to $r = 2 * 8 = 16$

i.e. $r = 16$

(e) Iteration – 5 : when $n = 1$, case 1 will be executed and the statement

$r = (n * n) / 2$; evaluates to $r = (1 * 1) / 2 = 1/2 = 0$ [int / int = int]

i.e. $r = 0$

5) Write a C program which will find sum of the following series using loops:

S1 = 50 + 46 + 42 + upto m1. (e.g.

upto 30)

S2 = 47 + 45 + 43 + m2 values. (e.g. 4 values i.e.

47+45+43+41)

Ans. #include<stdio.h>

#include<conio.h>

void main()

{

clrscr();

int choice,i;

printf("\nSeries Menu");

printf("\n-----");

printf("\n1. S1 = 50 + 46 + 42 + upto m1. (e.g. upto 30)");

printf("\n2. S2 = 47 + 45 + 43 + m2 values. (e.g. 4 values i.e. 47+45+43+41)");

printf("\n-----");

printf("\n\nEnter your choice [1/2] : ");

scanf("%d",&choice);

switch(choice)

{

case 1 : int s1=0,m1;

printf("\nEnter value of m1 : ");

scanf("%d",&m1);

for(i=50;i>=m1;i=i-4)

{

s1=s1+i;

}

printf("\nSum of 1st series : %d",s1);

break;

case 2 : int s2=0,j,m2;

```

        printf("\nEnter value of m2 : ");
        scanf("%d",&m2);
        for(i=1,j=47;i<=m2;i++,j=j-2)
        {
            s2=s2+j;
        }
        printf("\nSum of 2nd series : %d",s2);
        break;
    default : printf("\nInvalid choice");
}
getch();
}

```

6) Write a program in C to find sum of series:

$S = 2 - \frac{2}{5} + \frac{2}{5} - \frac{2}{7} + \dots + \frac{2}{n}$, where n is always odd.

[15]

[June-2010]

Ans.

```

#include<stdio.h>
#include<conio.h>
void main()
{
    clrscr();
    int i,n;
    float s=2.0;
    printf("\nS = 2 - 2/5 + 2/5 - 2/7 + ..... + 2/n");
    printf("\n\nEnter value of n : ");
    scanf("%d",&n);
    for(i=5;i<=n;i=i+2)
    {
        s=s+(float)2/i;
        s=s-(float)2/i;
    }
    printf("\nSum of given series : %.2f",s);
    getch();
}

```

7) What is nesting of loops? Write a program in C which accept 5-digit number from a user and calculate its sum.

[6] [Dec-2009]

Ans. (a) If there are many data items to be processed against a set of elements repeatedly, then a single for statement is not sufficient. So, we must use a nested for loop. If one loop statement is completely placed within the other, it is known as a nested loop statement or *nesting of loops*. The syntax of a nested while and nested for loops are as follows :

Nested while loop :

```
initialization-1 ;
while(test-condition 1)
{
    initialization-2;
    while(test-condition 2)
    {
        .....
        statement(s);
        .....
        Increment-2/decrement-2;
    }
    Increment-1/decrement-1;
}
```

Nested for loop :

```
for(initialization-1 ; test-condition 1; increment-1/decrement-1)
{
    for(initialization-2; test-condition 2; increment-2/decrement-2)
    {
        .....
        Statement(s);
        .....
    }
}
```

Rules for nested loop statement:

- a. One loop must completely be placed inside the other i.e. no overlapping
- b. Each loop must have different index variables.

(b) Program to find the sum of digits of 5 digit number is as follows :

```
#include<stdio.h>
#include<conio.h>
void main( )
{
    clrscr( );
    int n,dig,sum=0;
    printf("Enter any 5-digit number : ");
    scanf("%d",&n);
    while(n>0)
    {
        dig=n%10;
        sum=sum+dig;
        n=n/10;
    }
    printf("\nSum of digits : %d",sum);
    getch( );
}
```

8) Write a program to reverse a given number.

Ans.

```
#include<stdio.h>
#include<conio.h>
void main( )
{
    clrscr( );
    int n, dig,rev=0;
    printf("Enter number : ");
    scanf("%d",&n);
    while(n>0)
    {
        dig=n%10;
        rev=(rev*10)+dig;
        n=n/10;
    }
    printf("\nReverse number : %d",rev);
    getch( );
}
```

9) WAP to accept a number from keyboard and display whether the given number is palindrome or not.

Ans.

```
#include<stdio.h>
#include<conio.h>
```

```
void main( )
{
    clrscr( );
    long int n,m,rem,rev=0;
    printf("Enter number : ");
    scanf("%ld",&n);
    m=n;
    while(n>0)
    {
        rem=n%10;
        rev=(rev*10)+rem;
        n=n/10;
    }
    if(m==rev)
        printf("\n%ld is palindrome",m);
    else
        printf("\n%ld is not palindrome",m);
    getch( );
}
```

9) Write a program in C to calculate the factorial of a given number.

Ans.

```
#include<stdio.h>
#include<conio.h>
void main( )
{
    clrscr( );
    long int n,fact=1,i;
    printf("Enter number : ");
    scanf("%ld",&n);
    for(i=1;i<=n;i++)
        fact=fact*i;
    printf("\nFactorial of %ld is : %ld",n,fact);
    getch( );
}
```

10) Write a C program to find Exponent Power Series.

Ans.

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
void main()
{
    clrscr();
```

```
int i,j,x,n,sum=0;

printf("Enter value of x and n : ");
scanf("%d%d",&x,&n);
for(i=0;i<n;i++)
{
    sum=sum+pow(x,i);
}
printf("\nSum of given series : %d",sum);
getch();
}
```

Program : WAP to display the fibonacci series :

0 1 1 2 3 5 8 13 21 34

Ans.

```
#include<stdio.h>
#include<conio.h>
void main( )
{
    clrscr( );
    int pr=0,nx=1,nw,i;
    printf("%d\t%d",pr,nx);
    for(i=1;i<=8;i++)
    {
        nw=pr+nx;
        printf("\t%d",nw);
        pr=nx;
        nx=nw;
    }
    getch( );
}
```

12) WAP to accept a number and display whether it is prime or not

Ans.

```
#include<stdio.h>
#include<conio.h>
void main( )
```



```
{
clrscr( );

int n,i,f=0;
printf("Enter a number : ");
scanf("%d",&n);
for(i=2;i<n;i++)
{
if(n%i==0)
{
printf("\n%d is non prime",n);
f=1;
break;
}
}
if(f==0)
printf("\n%d is prime",n);
getch( );
}
```

11) Write a program to display first 20 prime numbers.

Ans.

```
#include<stdio.h>
#include<conio.h>
void main( )
{
clrscr( );
int n,i,flag,count;
printf("\nFirst 20 prime numbers are : \n");
for(n=1,count=1;count<=20;n++)
{
flag=0;
for(i=2;i<n;i++)
{
if(n%i==0)
{
flag=1;
break;
}
}
if(flag==0)
{
```

```
        printf("%d\t",n);
        count++;
    }
}
getch();
}
```

Program : WAP to accept and display a list of numbers till user wishes using do while loop.

Ans.

```
#include<stdio.h>
#include<conio.h>
void main()
{
    clrscr();
    int n;
    char choice;
    do
    {
        printf("Enter number : ");
        scanf("%d",&n);
        printf("%d",n);
        printf("\nLike to enter another number [y/n] : ");
        fflush(stdin);
        scanf("%c",&choice);
    }while(choice=='y' || choice=='Y');
    getch();
}
```

Program : WAP for a calculator which can perform all the basic mathematical operations acc. to choices given below :

- 1. Addition**
- 2. Subtraction**
- 3. Multiplication**
- 4. Division**
- 5. Power**
- 6. Square**
- 7. Square root**
- 8. Exit**

Ans. #include<stdio.h>
#include<conio.h>
#include<math.h>
#include<stdlib.h>

```
void main()
{
    clrscr();
    int a,b;
    int ch;
    char choice;
    do
    {
        clrscr();
        printf("  CALCULATOR\n");
        printf("-----\n");
        printf("1. Addition\n");
        printf("2. Subtraction\n");
        printf("3. Multiplication\n");
        printf("4. Division\n");
        printf("5. Power\n");
        printf("6. Square\n");
        printf("7. Square root\n");
        printf("8. Exit\n");
        printf("-----\n");
        printf("\nEnter your choice [1/2/3/4/5/6/7/8] : ");
        scanf("%d",&ch);
        if(ch<5)
        {
            printf("\nEnter two numbers : ");
            scanf("%d%d",&a,&b);
        }
        else
            if(ch==5)
            {
                printf("\nEnter number and its power : ");
                scanf("%d%d",&a,&b);
            }
            else
                if(ch==6 || ch==7)
                {
                    printf("\nEnter number : ");
                    scanf("%d",&a);
                }
            switch(ch)
            {
                case 1 : printf("\nSum of %d and %d is : %d",a,b,a+b);
                           break;
                case 2 : printf("\nDifference of %d and %d is : %d",a,b,a-b);
                           break;
                case 3 : printf("\nProduct of %d and %d is : %d",a,b,a*b);
```

```
        break;
    case 4 : printf("\nQuotient of %d and %d is : %d",a,b,a/b);
        break;
    case 5 : printf("\n%d raised to the power %d is : %.0f",a,b,pow(a,b));
        break;
    case 6 : printf("\nSquare of %d is : %d",a,a*a);
        break;
    case 7 : printf("\nSquare root of %d is : %.2f",a,sqrt(a));
        break;
    case 8 : exit(0);
    default : printf("\nInvalid choice");
}
printf("\n\nLike to perform another mathematical operation [y/n] : ");
fflush(stdin);
scanf("%c",&choice);
}while(choice=='y' || choice=='Y');
getch();
}
```